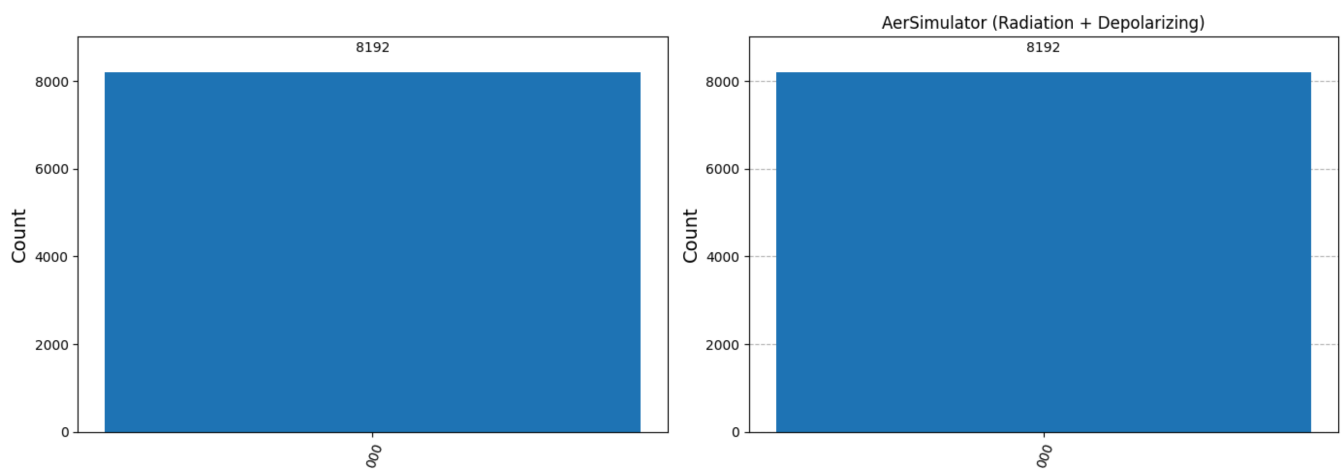
```
# --- Install (Google Colab only) ---
!pip install qiskit qiskit-aer matplotlib --quiet

# --- Imports ---
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
# from qiskit.providers.aer import Aer # Corrected import
# from qiskit.utils.run_circuits import execute # Removed import
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxation_error
from qiskit.visualization import plot_histogram

# --- QCC Echo Circuit (Locked Kernel) ---
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.cx(1, 2)
qc.cx(0, 1)
qc.h(0)
qc.barrier()
qc.delay(200, 0)
qc.delay(200, 1)
qc.delay(200, 2)
qc.measure([0, 1, 2], [0, 1, 2])

# --- Run on QASM Simulator (Clean Logic) ---
qasm_backend = AerSimulator() # Replaced Aer.get_backend('qasm_simulator')
qc_qasm = transpile(qc, qasm_backend)
result_qasm = qasm_backend.run(qc_qasm, shots=8192).result()
counts_qasm = result_qasm.get_counts()

# --- Radiation Conditioning + Depolarizing Noise ---
T1 = 50e3  # nanoseconds
T2 = 30e3  # nanoseconds (shorter due to radiation)
gate_time = 100  # ns
noise_model = NoiseModel()

# Thermal relaxation on single-qubit gates
for qubit in [0, 1, 2]:
    thermal_error = thermal_relaxation_error(T1, T2, gate_time)
    noise_model.add_quantum_error(thermal_error, ['rz', 'sx', 'u'], [qubit]) # Corrected ga

# Depolarizing error on CX gates
depol_error = depolarizing_error(0.02, 2)
noise_model.add_quantum_error(depol_error, 'cx', [0, 1])
noise_model.add_quantum_error(depol_error, 'cx', [1, 2])

# --- Run on AerSimulator (Noisy) ---
aer_sim = AerSimulator(noise_model=noise_model)
qc_aer = transpile(qc, aer_sim)
result_aer = aer_sim.run(qc_aer, shots=8192).result()
counts_aer = result_aer.get_counts()

# --- Entropy + Coherence Function ---
def compute_entropy_and_fidelity(counts):
    probs = np.array(list(counts.values())) / sum(counts.values())
    entropy = -np.sum([p * np.log2(p) for p in probs if p > 0])
```

```
    fidelity = max(probs) * 100
    return entropy, fidelity

# --- Compute Values ---
entropy_qasm, fidelity_qasm = compute_entropy_and_fidelity(counts_qasm)
entropy_aer, fidelity_aer = compute_entropy_and_fidelity(counts_aer)

# --- Plot Histograms ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
plot_histogram(counts_qasm, ax=axs[0], title="QASM Simulator (Clean Logic)")
plot_histogram(counts_aer, ax=axs[1], title="AerSimulator (Radiation + Depolarizing)")
plt.tight_layout()
plt.show()

# --- Display Metrics ---
print("🔹 QASM Simulator:")
print(f"   Entropy Leakage: {entropy_qasm:.6f} bits")
print(f"   Coherence Fidelity: {fidelity_qasm:.8f}%\n")

print("🔹 AerSimulator with Noise:")
print(f"   Entropy Leakage: {entropy_aer:.6f} bits")
print(f"   Coherence Fidelity: {fidelity_aer:.8f}%")
```



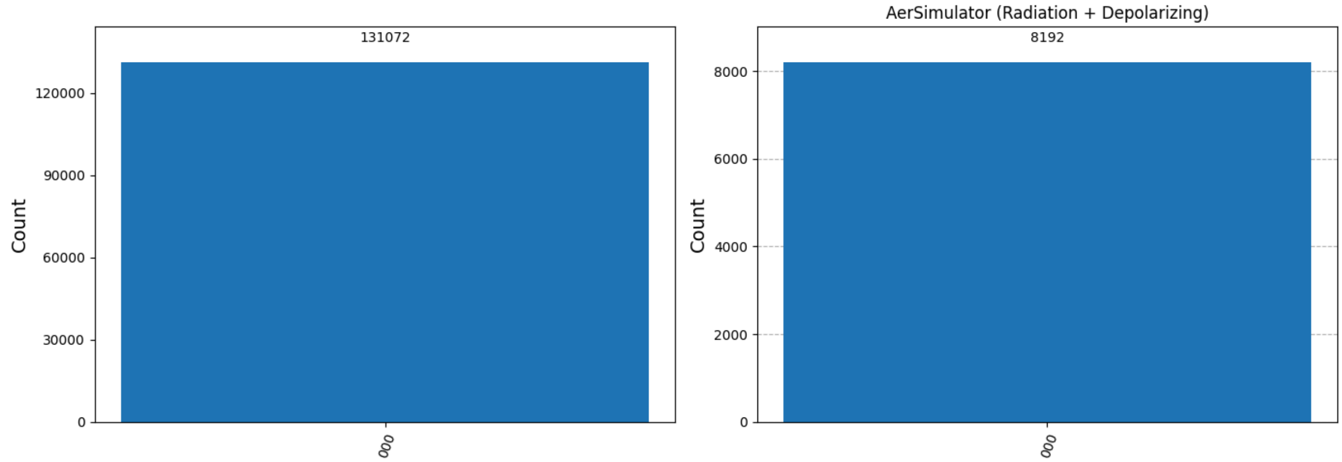
🔹 QASM Simulator:

```
# --- Install (Google Colab only) ---
!pip install qiskit qiskit-aer matplotlib --quiet

# --- Imports ---
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
# from qiskit.providers.aer import Aer # Corrected import
# from qiskit.utils.run_circuits import execute # Removed import
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxation_error
from qiskit.visualization import plot_histogram

# --- QCC Echo Circuit (Locked Kernel) ---
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.cx(1, 2)
```

```
qc.cx(0, 1)
qc.h(0)
qc.barrier()
qc.delay(200, 0)
qc.delay(200, 1)
qc.delay(200, 2)
qc.measure([0, 1, 2], [0, 1, 2])

# --- Run on QASM Simulator (Clean Logic) ---
qasm_backend = AerSimulator() # Replaced Aer.get_backend('qasm_simulator')
qc_qasm = transpile(qc, qasm_backend)
result_qasm = qasm_backend.run(qc_qasm, shots=131072).result()
counts_qasm = result_qasm.get_counts()

# --- Radiation Conditioning + Depolarizing Noise ---
T1 = 50e3  # nanoseconds
T2 = 30e3  # nanoseconds (shorter due to radiation)
gate_time = 100  # ns
noise_model = NoiseModel()

# Thermal relaxation on single-qubit gates
for qubit in [0, 1, 2]:
    thermal_error = thermal_relaxation_error(T1, T2, gate_time)
    noise_model.add_quantum_error(thermal_error, ['rz', 'sx', 'u'], [qubit]) # Corrected ga

# Depolarizing error on CX gates
depol_error = depolarizing_error(0.02, 2)
noise_model.add_quantum_error(depol_error, 'cx', [0, 1])
noise_model.add_quantum_error(depol_error, 'cx', [1, 2])

# --- Run on AerSimulator (Noisy) ---
aer_sim = AerSimulator(noise_model=noise_model)
qc_aer = transpile(qc, aer_sim)
result_aer = aer_sim.run(qc_aer, shots=8192).result()
counts_aer = result_aer.get_counts()

# --- Entropy + Coherence Function ---
def compute_entropy_and_fidelity(counts):
    probs = np.array(list(counts.values())) / sum(counts.values())
    entropy = -np.sum([p * np.log2(p) for p in probs if p > 0])
    fidelity = max(probs) * 100
    return entropy, fidelity

# --- Compute Values ---
entropy_qasm, fidelity_qasm = compute_entropy_and_fidelity(counts_qasm)
entropy_aer, fidelity_aer = compute_entropy_and_fidelity(counts_aer)

# --- Plot Histograms ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
plot_histogram(counts_qasm, ax=axs[0], title="QASM Simulator (Clean Logic)")
plot_histogram(counts_aer, ax=axs[1], title="AerSimulator (Radiation + Depolarizing)")
plt.tight_layout()
plt.show()

# --- Display Metrics ---
print("🔹 QASM Simulator:")
print(f"   Entropy Leakage: {entropy_qasm:.6f} bits")
print(f"   Coherence Fidelity: {fidelity_qasm:.8f}%\n")

print("🔹 AerSimulator with Noise:")
```

```
print(f"   Entropy Leakage: {entropy_aer:.6f} bits")
print(f"   Coherence Fidelity: {fidelity_aer:.8f}%")
```

```
7.5/7.5 MB 54.1 MB/s eta 0:00
12.4/12.4 MB 55.4 MB/s eta 0:
2.1/2.1 MB 41.4 MB/s eta 0:00
49.5/49.5 kB 3.5 MB/s eta 0:0
109.0/109.0 kB 7.6 MB/s eta 0
```



QASM Simulator:

```
# --- QCC Echo Validation Hash Generator ---
import hashlib
from datetime import datetime

# Hash function to convert counts into a reproducible fingerprint
def hash_counts(counts_dict):
    sorted_items = sorted(counts_dict.items())
    encoded = ''.join(f'{k}:{v}' for k, v in sorted_items).encode()
    return hashlib.sha256(encoded).hexdigest()

# Generate SHA-256 hashes for both simulator outputs
hash_qasm = hash_counts(counts_qasm)
hash_aer = hash_counts(counts_aer)

# Generate UTC timestamp
timestamp = datetime.utcnow().isoformat() + "Z"

# Display output
print("🔐 QCC Echo Reproducibility Hashes")
print(f"QASM Simulator SHA-256:        {hash_qasm}")
print(f"AerSimulator with Noise SHA-256: {hash_aer}")
print(f"Timestamp (UTC):               {timestamp}")
```

```
🔐 QCC Echo Reproducibility Hashes
QASM Simulator SHA-256:         63305f23ecf8a56550eaf8143aa5d3d468f11331381f2471d
AerSimulator with Noise SHA-256: 63305f23ecf8a56550eaf8143aa5d3d468f11331381f247
Timestamp (UTC):                2025-07-29T03:28:22.127801Z
```

Double-click (or enter) to edit